

IEEE Inter-Society Code Debugging Challenge — SET A Questions

Instructions for participants: Each problem has a working idea but **buggy code**. Find and fix the bugs so the program passes all hidden test cases. Use the language given for each problem. Keep the original structure, because fully rewritten solutions may be disqualified. The bug count is a hint.

Moderate = 50 pts · Hard = 100 pts

PART A — MODERATE (15 × 50 pts)

M1. Single Neuron Activation · Python · IEEE CIS · AI&DS/CSE · 3 bugs

Concept: Weighted sum, sigmoid activation function.

Compute the output of a neuron with sigmoid activation: $\sigma(\sum w_i x_i + b)$, printed with exactly 4 decimals.

Input: n ; then n weights; then n inputs; then bias b (all integers, $|values| \leq 10$, $1 \leq n \leq 100$)

Sample: 2 / 1 2 / 1 1 / 0 → 0.9526

```
import math
n = int(input())
w = list(map(int, input().split()))
x = list(map(int, input().split()))
b = int(input())
z = 0
for i in range(1, n):
    z += w[i] * x[i]
z + b
print(round(1 / (1 - math.exp(-z)), 4))
```

M2. First Occurrence · C++ · Computer Society · CSE · 3 bugs

Concept: Binary search, lower bound / first occurrence.

Given a sorted array, print the 0-based index of the **first** occurrence of t , or -1 .

Input: n t , then n sorted integers ($1 \leq n \leq 10^5$)

Sample: 6 3 / 1 3 3 3 5 7 → 1

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, t; cin >> n >> t;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int lo = 0, hi = n, ans = -1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) { ans = mid; lo = mid + 1; }
        else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    cout << ans << endl;
}
```

M3. Caesar Cipher · Java · Security · Cyber · 3 bugs

Concept: Caesar cipher, modular arithmetic on characters.

Shift every letter by k (k may be negative). Keep the case. Non-letters stay unchanged.

Input: k ($|k| \leq 1000$) on line 1, text on line 2

Sample: 3 / Hello, World! → Khoor, Zruog!

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int k = Integer.parseInt(sc.nextLine().trim());
        String s = sc.nextLine();
        k = k % 26;
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isLowerCase(c))
                sb.append((char) ((c - 'a' + k) % 26));
            else if (Character.isUpperCase(c))
                sb.append((char) ('A' + (c - 'a' + k) % 26));
            else
                sb.append(c);
        }
        System.out.println(sb);
    }
}
```

M4. Moving Average Filter · Python · Signal Processing Society · ECE · 3 bugs

Concept: Moving-average (FIR low-pass) filter, sliding window sum.

Print the average of every window of size k ($n-k+1$ values, 2 decimals, space-separated).

Input: n k , then n integers ($1 \leq k \leq n \leq 10^5$)

Sample: 5 3 / 1 2 3 4 5 → 2.00 3.00 4.00

```

n, k = map(int, input().split())
a = list(map(int, input().split()))
s = sum(a[:k])
res = [s / k]
for i in range(k, n - 1):
    s += a[i] - a[i - k + 1]
    res.append(s // k)
print(" ".join(f"{v:.2f}" for v in res))

```

M5. Hamming Distance · C++ · Circuits & Systems · VLSI/ECE · 2 bugs

Concept: XOR, Hamming distance, bit shifting.

Print the number of bit positions where `x` and `y` differ.

Input: `x y` ($0 \leq x, y < 2^{31}$)

Sample: `1 4 → 2`

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    long long x, y; cin >> x >> y;
    long long d = x & y;
    int c = 0;
    while (d > 0) {
        c += d & 1;
        d << 1;
    }
    cout << c << endl;
}

```

M6. Sentence Palindrome · Java · Computer Society · CSE · 3 bugs

Concept: Two-pointer technique, string normalization.

Ignore non-alphanumeric characters and case. Print `YES` if the line is a palindrome, otherwise `NO`.

Input: one line (length $\leq 10^5$)

Sample: `A man, a plan, a canal: Panama → YES`

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (!Character.isLetterOrDigit(s.charAt(i))) i++;
            if (!Character.isLetterOrDigit(s.charAt(j))) j--;
            if (s.charAt(i) != s.charAt(j)) { System.out.println("NO"); return; }
            i++; j--;
        }
        System.out.println("YES");
    }
}

```

M7. Matrix Multiplication · Python · IEEE CIS · AI&DS · 3 bugs

Concept: Matrix multiplication, list aliasing in Python.

Multiply A ($n \times m$) by B ($m \times p$). Print C row by row, space-separated.

Input: `n m p`, then `n` rows of A, then `m` rows of B (≤ 50 each)

Sample: `2 2 2 / 1 2 / 3 4 / 5 6 / 7 8` → `19 22 / 43 50`

```
n, m, p = map(int, input().split())
A = [list(map(int, input().split())) for _ in range(n)]
B = [list(map(int, input().split())) for _ in range(m)]
C = [[0] * n] * n
for i in range(n):
    for j in range(p):
        for k in range(m):
            C[i][j] += A[i][k] * B[j][k]
for row in C:
    print(*row)
```

M8. Decimal to Binary · C++ · Circuits & Systems · VLSI · 4 bugs

Concept: Base conversion (decimal → binary), char arithmetic.

Print the binary representation of `n` without leading zeros.

Input: `n` ($0 \leq n \leq 10^9$)

Sample: `10` → `1010`, `0` → `0`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    string s;
    while (n > 1) {
        s += char(n % 2);
        n /= 2;
    }
    cout << s << endl;
}
```

M9. Balanced Brackets · Java · Computer Society · CSE · 3 bugs

Concept: Stack, bracket matching.

For each of the `t` strings made of `()[]{}` , print `YES` if it is balanced, otherwise `NO`.

Input: `t`, then `t` lines (each non-empty, length $\leq 10^5$)

Sample: `3 / {[()]} / ([)] / ((` → `YES / NO / NO`

```

import java.util.*;
public class Main {
    static boolean ok(String s) {
        Deque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') st.push(c);
            else {
                char top = st.pop();
                if ((c == ')' && top != '(') || (c == ']' && top != '[') || (c == '}' &&
top != '{'))
                    return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = Integer.parseInt(sc.nextLine().trim());
        while (t-- > 0) System.out.println(ok(sc.nextLine().trim()) ? "YES" : "NO");
    }
}

```

M10. 1-Nearest-Neighbour Classifier · Python · IEEE CIS · AI&DS · 3 bugs

Concept: k-Nearest Neighbours, Euclidean distance.

Classify the query point with the label of the closest training point (Euclidean distance). On a tie, use the earliest point.

Input: `n d`; `n` lines with `d` integers and a label; one line with `d` integers (the query)

Sample: `3 2 / 0 0 A / 5 5 B / 1 2 A / 4 4 → B`

```

n, d = map(int, input().split())
train = []
for _ in range(n):
    parts = input().split()
    train.append((list(map(int, parts[:d])), parts[d]))
query = list(map(int, input().split()))
best, label = 0, None
for pt, l in train:
    dist = sum(abs(p - q) for p, q in zip(pt, query)) ** 0.5
    if dist > best:
        best, label = dist, l
print(label)

```

M11. Prime Counter (RSA prep) · C++ · Security · Cyber · 4 bugs

Concept: Sieve of Eratosthenes.

Count the primes $\leq N$ using the Sieve of Eratosthenes.

Input: `N` ($1 \leq N \leq 10^7$)

Sample: `10 → 4`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<bool> isP(n, true);
    for (int i = 2; (long long)i * i < n; i++)
        if (isP[i])
            for (int j = i * i; j < n; j += i) isP[j] = false;
    int c = 0;
    for (int i = 0; i <= n; i++) if (isP[i]) c++;
    cout << c << endl;
}
```

M12. Fibonacci mod $1e9+7$ · Java · Computer Society · CSE · 3 bugs

Concept: Iterative DP, modular arithmetic, overflow.

Print $F(n) \bmod 1\,000\,000\,007$, where $F(0)=0$ and $F(1)=1$.

Input: n ($0 \leq n \leq 10^6$)

Sample: 10 → 55

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int MOD = 1_000_000_007;
        int n = new Scanner(System.in).nextInt();
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            int c = a + b % MOD;
            a = b; b = c;
        }
        System.out.println(b);
    }
}
```

M13. Even Parity Generator · Python · Signal Processing / Comm. · ECE · 3 bugs

Concept: Parity bits, error detection.

Append a parity bit to each binary word so that the total number of 1s is **even**.

Input: t , then t binary strings

Sample: 2 / 1011 / 1001 → 10111 / 10010

```
t = int(input())
for _ in range(t):
    s = input().strip()
    ones = s.count(1)
    p = 1 if ones % 2 == 0 else 0
    print(s + p)
```

M14. Maximum Subarray Sum · C++ · IEEE CIS · AI&DS · 3 bugs

Concept: Kadane's algorithm (DP), overflow.

Print the maximum sum of a non-empty contiguous subarray.

Input: n , then n integers ($n \leq 10^5$, $|a_i| \leq 10^9$)

Sample: 3 / -3 -1 -2 → -1

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    int best = 0, cur = 0;
    for (int x : a) {
        cur = max(x, cur + x);
        best = max(best, x);
    }
    cout << best << endl;
}
```

M15. Password Strength Checker · Java · Security · Cyber · 4 bugs

Concept: Input validation, boolean logic.

A password is **STRONG** if it has length ≥ 8 **and** at least one uppercase letter, one lowercase letter, one digit and one special character from `!@#$%^&*`. Otherwise it is **WEAK**.

Input: one line (no spaces)

Sample: Ieee@2026 → STRONG, password → WEAK

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String p = new Scanner(System.in).next();
        String specials = "!@#$%^&*";
        boolean up = false, low = false, dig = false, sp = false;
        for (char c : p.toCharArray()) {
            if (Character.isLowerCase(c)) up = true;
            else if (Character.isLowercase(c)) low = true;
            else if (Character.isDigit(c)) dig = true;
            else if (specials.indexOf(c) > 0) sp = true;
        }
        boolean strong = p.length() > 8 && (up || low || dig || sp);
        System.out.println(strong ? "STRONG" : "WEAK");
    }
}
```

PART B — HARD (10 × 100 pts)

H1. Shortest Network Latency (Dijkstra) · C++ · Computer Society · CSE · 4 bugs

Concept: Dijkstra's shortest path, min-heap, lazy deletion.

An **undirected** network has n routers and m links with latencies. Print the shortest latency from router 1 to every router (-1 if it can't be reached), space-separated.

Input: n m , then m lines u v w ($n, m \leq 2 \cdot 10^5$, $w \leq 10^9$)

Sample: 4 3 / 1 2 5 / 2 3 7 / 1 3 20 → 0 5 12 -1

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<pair<int,int>>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v, w; cin >> u >> v >> w;
        g[u].push_back({v, w});
    }
    const int INF = INT_MAX;
    vector<int> dist(n + 1, INF);
    priority_queue<pair<int,int>> pq;
    dist[1] = 0; pq.push({0, 1});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    for (int i = 1; i <= n; i++) cout << (dist[i] == INF ? -1 : dist[i]) << " \n"[i == n];
}
```

H2. Linear Regression by Gradient Descent · Python · IEEE CIS · AI&DS · 3 bugs

Concept: Gradient descent, MSE loss, linear regression.

Fit $y = w \cdot x + b$ with batch gradient descent on the MSE loss (gradient $(1/n) \sum \text{err} \cdot x$, $(1/n) \sum \text{err}$). Start from $w=b=0$ and run exactly T iterations with learning rate lr , updating w and b **simultaneously**. Print w b with 4 decimals.

Input: n T lr , then n lines x y

Sample: 3 5000 0.05 / 1 3 / 2 5 / 3 7 → 2.0000 1.0000

```
n, T, lr = input().split()
n, T, lr = int(n), int(T), float(lr)
xs, ys = [], []
for _ in range(n):
    x, y = map(float, input().split())
    xs.append(x); ys.append(y)
w = b = 0.0
dw = db = 0.0
for _ in range(T):
    for i in range(n):
        err = (w * xs[i] + b) - ys[i]
        dw += err * xs[i]
        db += err
    w += lr * dw / n
    b -= lr * db
print(f"{w:.4f} {b:.4f}")
```

H3. LRU Cache · Java · Computer Society · CSE · 3 bugs

Concept: LRU cache, LinkedHashMap access order.

Simulate an LRU cache of capacity `c`. `GET k` prints the value or `-1`, and counts as a use. `PUT k v` inserts or updates, and also counts as a use. When the cache is full, evict the least recently used key.

Input: `c q`, then `q` operations

Sample: `2 6 / PUT 1 1 / PUT 2 2 / GET 1 / PUT 3 3 / GET 2 / GET 1 → 1 / -1 / 1`

```
import java.util.*;
public class Main {
    static class LRU extends LinkedHashMap<Integer, Integer> {
        int cap;
        LRU(int cap) { super(16, 0.75f, false); this.cap = cap; }
        protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) { return
size() >= cap; }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int c = sc.nextInt(), q = sc.nextInt();
        LRU cache = new LRU(c);
        StringBuilder out = new StringBuilder();
        while (q-- > 0) {
            String op = sc.next();
            if (op.equals("GET")) {
                int v = cache.get(sc.nextInt());
                out.append(v).append('\n');
            } else cache.put(sc.nextInt(), sc.nextInt());
        }
        System.out.print(out);
    }
}
```

H4. FIR Filter / Discrete Convolution · C++ · Signal Processing Society · ECE · 3 bugs

Concept: Discrete convolution, FIR filtering.

Compute $y[n] = \sum_k h[k] \cdot x[n-k]$ (full linear convolution, length $N+M-1$).

Input: `N M`, then `N` values of `x`, then `M` values of `h` ($N, M \leq 2000$, $|\text{values}| \leq 10^6$)

Sample: `3 2 / 1 2 3 / 1 1 → 1 3 5 3`

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int N, M; cin >> N >> M;
    vector<int> x(N), h(M);
    for (auto &v : x) cin >> v;
    for (auto &v : h) cin >> v;
    int L = N + M;
    vector<int> y(L, 0);
    for (int n = 0; n < L; n++)
        for (int k = 0; k < M; k++)
            if (n - k > 0 && n - k < N) y[n] += h[k] * x[n - k];
    for (int i = 0; i < L; i++) cout << y[i] << " \n"[i == L - 1];
}
```

H5. RSA Decryption · Python · Security · Cyber · 4 bugs

Concept: RSA, modular inverse (Extended Euclid), fast exponentiation.

Given primes `p q`, public exponent `e` and ciphertext `c`, compute $d = e^{-1} \bmod \phi(n)$ and print $m = c^d \bmod n$. Do **not** use the built-in `pow` with 3 arguments. Fix the helper functions instead.

Input: `p q e c` ($p, q < 10^9$)

Sample: `61 53 17 2790` → `65`

```
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = egcd(b, a % b)
    return g, y, x - (a // b) * y

def inv(a, m):
    g, x, _ = egcd(a, m)
    return x

def power(b, e, m):
    r = 1
    b %= m
    while e > 0:
        if e & 1:
            r = r * b % m
        b = b * b
        e >> 1
    return r

p, q, e, c = map(int, input().split())
n = p * q
phi = p * q - 1
d = inv(e, phi)
print(power(c, d, n))
```

H6. Course Scheduler (Topological Sort) · Java · Computer Society · CSE · 3 bugs

Concept: Topological sort (Kahn's algorithm), cycle detection.

There are n courses and m rules $u\ v$ meaning u must be taken before v . Print the **lexicographically smallest** valid order, or **CYCLE** if no order exists.

Input: $n\ m$, then m lines $u\ v$ (1-indexed, $n, m \leq 10^5$)

Sample: 4 3 / 1 2 / 1 3 / 3 4 → 1 2 3 4

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), m = sc.nextInt();
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i <= n; i++) g.add(new ArrayList<>());
        int[] indeg = new int[n + 1];
        for (int i = 0; i < m; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            g.get(u).add(v);
            indeg[v]++;
        }
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
        List<Integer> order = new ArrayList<>();
        while (!q.isEmpty()) {
            int u = q.pop();
            order.add(u);
            for (int v : g.get(u)) if (--indeg[v] == 0) q.push(v);
        }
        StringBuilder sb = new StringBuilder();
        for (int x : order) sb.append(x).append(' ');
        System.out.println(sb.toString().trim());
    }
}
```

H7. Binary Ripple-Carry Adder · C++ · Circuits & Systems · VLSI · 4 bugs

Concept: Ripple-carry adder, binary addition.

Add two binary strings, the way a ripple-carry adder does, and print the binary sum.

Input: two binary strings (length $\leq 10^5$, no leading zeros unless the string is "0")

Sample: 1011 111 → 10010

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string a, b; cin >> a >> b;
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    string res;
    while (i >= 0 && j >= 0) {
        int sum = carry;
        if (i >= 0) sum += a[i--];
        if (j >= 0) sum += b[j--];
        res.push_back('0' + sum / 2);
        carry = sum % 2;
    }
    cout << res << endl;
}
```

H8. 1-D K-Means Clustering · Python · IEEE CIS · AI&DS · 4 bugs

Concept: K-means clustering (Lloyd's algorithm).

Start with the **first k points** as the initial centroids and run exactly **T** iterations:

1. Assign every point to the nearest centroid (on a tie, the lower index).
2. Recompute **all** centroids from the new assignment. An empty cluster keeps its old centroid.

Print the final centroids sorted ascending, 2 decimals, space-separated.

Input: **n k T**, then **n** integers

Sample: 6 2 10 / 1 2 3 10 11 12 → 2.00 11.00

```
n, k, T = map(int, input().split())
pts = list(map(int, input().split()))
cent = [float(v) for v in pts[:k]]
for _ in range(T):
    clusters = [[]] * k
    for p in pts:
        best = 0
        for c in range(1, k):
            if abs(p - cent[c]) < abs(p - cent[best]):
                best = c
        clusters[best].append(p)
        cent[best] = sum(clusters[best]) / len(clusters[best])
    for c in range(k):
        cent[c] = sum(clusters[c]) / len(clusters[c])
print(" ".join(f"{v:.2f}" for v in sorted(cent)))
```

H9. Peak Detector (Sliding Window Maximum) · Java · Signal Processing Society · ECE · 3 bugs

Concept: Sliding window maximum, monotonic deque.

For each window of size **k**, print the maximum signal value. Use the $O(n)$ deque method.

Input: **n k**, then **n** integers ($n \leq 10^6$)

Sample: 8 3 / 1 3 -1 -3 5 3 6 7 → 3 3 5 5 6 7

```

import java.util.*;
import java.io.*;
public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken()), k = Integer.parseInt(st.nextToken());
        int[] a = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());
        Deque<Integer> dq = new ArrayDeque<>();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            if (!dq.isEmpty() && dq.peekFirst() < i - k) dq.pollFirst();
            while (!dq.isEmpty() && a[dq.peekLast()] > a[i]) dq.pollLast();
            dq.addLast(i);
            if (i >= k) sb.append(a[dq.peekFirst()]).append(' ');
        }
        System.out.println(sb.toString().trim());
    }
}

```

H10. Edit Distance (Spell-check / DNA) · C++ · IEEE CIS · CSE/AI&DS · 5 bugs

Concept: Dynamic programming, Levenshtein edit distance.

Print the minimum number of insertions, deletions and substitutions needed to turn `s` into `t`.

Input: two strings `s t` (length ≤ 5000)

Sample: kitten sitting → 3

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    string s, t; cin >> s >> t;
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n, vector<int>(m, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            int cost = (s[i] == t[j]) ? 1 : 0;
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i - 1][j - 1] + cost);
        }
    cout << dp[n][m] << endl;
}

```